

NanoPoreSeq

NanoPoreSeq — Review report, procedures and corrections

Simulator of Nanopore BAMs of CNS tumours for testing Rapid-CNS2_nf

Prepared for Pierre Bady · 26 October 2026 · 12 scripts reviewed in five passes

This project is developed with the help of AI (Claude, Anthropic), under active review by Pierre Bady.

Document outline

1. Summary
2. File layout
3. Installation and requirements
4. Usage procedures
5. Detailed workings
6. Description of the scripts and their functions
7. Errors found and corrections
8. Rationale for the choices
9. Tests performed
10. Limitations and recommendations

1. Summary

The 12 scripts of the project were reviewed line by line in five passes, corrected, then grouped into three folders by use: simulation, running Rapid-CNS2 and the self-contained USB stick. Each correction was checked by a test run on simulated BAMs; the R scripts were run on the real data supplied with Rapid-CNS2 (Capper et al.).

The most important error was in bedmethyl_from_bam.py: methylation calls from reads on the – strand were placed at wrong positions (95% outside CpGs), so half of the data passed to the classification was noise. Two other defects had a practical impact: the memory use of the simulator (one cohort individual killed for lack of memory) and an MGMT coverage too low (~4–7×) for the 10× threshold used by the launchers.

After correction, the measurements on simulated BAMs recover the expected values (MGMT methylation 0.89 for 0.88 expected; transplanted HM450 probes 0.03 / 0.91 for 0.04 / 0.92; 19q codeletion 0.50; MGMT coverage 18–22×), peak memory is divided by 2.7, a 3-individual test cohort is generated without failure and the R classification of a simulated gbm BAM gives the expected class, GBM_RTK_II.

This report then describes, step by step, the installation and usage procedures (section 4), the internal workings of each step (section 5), the functions of each script (section 6), then the corrections, choices, tests and limitations.

2. File layout

The scripts are grouped by use. Within a folder, the scripts call each other by their relative path: the files of one folder must be kept together.

Folder	Use	Files
`simulation/`	Simulate BAMs, check them, simulate cohorts and classify their methylation	`simulate_glioma_hg38.py, verify_bam.py, build_cohort.py, bedmethyl_from_bam.py, methylation_classification_local.R, 01_export_probe_targets.R, 02_export_cohort_probe_targets.R, run_full_mnp_classification.sh, simnanocns_gui_en.py`
`rapidcns2/`	Run the Rapid-CNS2_nf pipeline; prepare real data	`rapidcns2_gui_en.py, align_fastq_rapidcns2_en.sh`
`usb/`	Build a self-contained USB stick (pipeline, containers, references, interface)	`make_usb_stick_en.sh, prepare_usb_en.sh`
`docs/`	User guides and review reports	`NanoPoreSeq_guide_utilisateur_FR.docx, NanoPoreSeq_user_guide_EN.docx, NanoPoreSeq_rapport_FR.docx, NanoPoreSeq_report_EN.docx`

Project tree

```
NanoPoreSeq/
├── README.md                                overview of the folders
├── simulation/                              simulate, check, classify
│   ├── simulate_glioma_hg38.py              simulates a BAM
│   ├── verify_bam.py                        checks a BAM
│   ├── build_cohort.py                      simulates a cohort
│   ├── bedmethyl_from_bam.py                BAM → bedMethyl
│   ├── methylation_classification_local.R   classifies the methylation
│   ├── 01_export_probe_targets.R            1 real profile per class
│   ├── 02_export_cohort_probe_targets.R     N real profiles per class
```

```
│   ├── run_full_mnp_classification.sh        complete chain
│   └── simnanocns_gui_en.py                  Streamlit interface
├── rapidcns2/                                run Rapid-CNS2_nf
│   ├── rapidcns2_gui_en.py                   Tkinter interface
│   └── align_fastq_rapidcns2_en.sh           FASTQ → BAM (real data)
├── usb/
│   ├── make_usb_stick_en.sh                  formats and creates the USB stick (all-in-one)
│   └── prepare_usb_en.sh                     copies the content onto the stick
└── docs/                                     guides and reports (FR, EN; .docx and .pdf)
```

The trees of the folders produced by the scripts (sample, cohort, classification, Rapid-CNS2 results, USB stick) are described in the user guide, section 8.5.

Overall flow

1. Simulation: simulate_glioma_hg38.py (one sample) or build_cohort.py (N samples) produces hg38 BAMs with MM/ML tags and a ground-truth file.
2. Check: verify_bam.py compares the BAM with the ground truth.
3. Local classification (optional): bedmethyl_from_bam.py then methylation_classification_local.R give the methylation class expected from Rapid-CNS2.
4. Pipeline test: rapidcns2_gui_en.py (or the USB stick) runs Rapid-CNS2_nf on the BAMs; the report produced is compared with the ground truth.

3. Installation and requirements

3.1 Software

1. Python 3 with pysam and numpy (simulation, verification, bedMethyl).

   ```
   python3 -m pip install pysam numpy
   ```

2. Simulation interface: Streamlit (version 1.37 or later) and pandas.

   ```
   python3 -m pip install "streamlit>=1.37" pandas
   ```

3. Rapid-CNS2 interface: tkinter.

   ```
   sudo apt install python3-tk
   ```

4. Classification: R with the optparse, ranger, matrixStats, data.table, glmnet and GenomicRanges (Bioconductor) packages.

   ```
   Rscript -e
   'install.packages(c("optparse","ranger","matrixStats","data.table","glmnet","BiocManager")
   )'
   Rscript -e 'BiocManager::install("GenomicRanges")'
   # or, on Ubuntu (method used for the tests):
   sudo apt install r-base-core r-cran-optparse r-cran-ranger r-cran-matrixstats \
       r-cran-data.table r-cran-glmnet r-bioc-genomicranges
   ```

5. Pipeline: Nextflow, Java 17 or later, Docker or Apptainer/Singularity. Real data: dorado and samtools. USB stick: rsync and curl.

3.2 Data

1. The data/ folder of Rapid-CNS2_nf, which provides the panel, the regions and the training data.

   ```
   git clone https://github.com/areebapatel/Rapid-CNS2_nf
   export DATA=$PWD/Rapid-CNS2_nf/data
   ```

2. Files used: NPHD_panel_hg38.bed (panel), mgmt_hg38.bed (MGMT promoter), genes.bed (EGFR, CDKN2A/B), refGene.txt.gz (ATRX and CIC sites); for the classification: top_probes_hm450.Rdata, HM450.hg38.manifest.gencode.v22.Rdata, capper_top_100k_betas_binarised.Rdata.
3. An hg38 FASTA whose contigs are named chr1… (UCSC) or NC_000001.11… (NCBI RefSeq). Its .fai index is created automatically on first use if the folder is writable.

```
export REF=/export/scratch3/data/Annotations/UCSC_HG38/hg38.fa
```

3.3 Installation check

1. The command-line scripts print their help with --help; a first quick test on a single chromosome with the stand-in sequence (--standin) checks the chain without a real reference (less than one minute for chr10 or chr17 in our tests).

```
cd NanoPoreSeq/simulation
python3 simulate_glioma_hg38.py --help
python3 simulate_glioma_hg38.py --profile gbm --standin --data-dir $DATA --out test_gbm --
chroms chr10
```

4. Usage procedures

Each procedure states its aim, the steps with the commands, the files produced and the points to check. The commands assume that the DATA and REF variables are set (section 3) and that you are in the folder shown.

4.1 Simulate one sample and check it

Aim: Produce a BAM of a given profile (gbm, astro or oligo), then check that it contains the simulated alterations. `[simulation/]`

Steps

1. Choose the profile and the tumour purity. The VAF of heterozygous variants is purity/2; TERT C228T of gbm stays fixed at 0.80.
2. Run a quick test on one or two chromosomes with the stand-in sequence.

```
python3 simulate_glioma_hg38.py --profile oligo --standin --data-dir $DATA \
    --out test_oligo --chroms chr2,chr19
python3 verify_bam.py --bam test_oligo/SIM_OLIGO_HG38.bam --data-dir $DATA --standin
```

3. Run the full simulation on the real hg38 (chr1–22 and chrX by default). The script first checks that the reference base of each hotspot is the one expected in hg38.

```
python3 simulate_glioma_hg38.py --profile gbm --ref $REF --data-dir $DATA \
    --out sim_gbm --purity 0.8 --seed 2026
```

4. Check the BAM. With --standin, give the same --seed as the simulation.

```
python3 verify_bam.py --bam sim_gbm/SIM_GBM_HG38.bam --data-dir $DATA --ref $REF
```

Main options

Option	Default	Effect
`--profile`	—	gbm, astro or oligo (required)
`--ref / --standin`	—	Real hg38 FASTA, or synthetic test sequence (one of the two)
`--data-dir`	—	data/ folder of Rapid-CNS2_nf (required)
`--out / --sample-id`	sim_glioma / SIM_<PROFILE>_HG38	Output folder and sample name
`--purity`	0.80	Tumour purity; heterozygous VAF = purity/2

Option	Default	Effect
`--ontarget-depth`	8	Depth per panel region (effective coverage ~2.5 times higher)
`--offtarget-depth`	2	Off-panel depth (used for CNV calling)
`--mgmt-depth-factor`	4	Dedicated depth on the MGMT promoter = factor × on-target
`--cdkn2a-del`	no	astro: adds the CDKN2A/B homozygous deletion (grade 4)
`--cnv-jitter-sd`	0	Multiplicative noise on the copy ratios
`--chroms`	chr1–22, chrX	Restricts the chromosomes (quick tests)
`--seed`	2026	Seed: same seed and same options, same BAM
`--mnp-targets / --mnp-depth / --mnp-window`	— / 6 / 150	Real methylation profile at the HM450 probes, dedicated depth and half-window
`--skip-ref-check`	no	Skips the reference base check

Files produced

- <out>/<id>.bam and <id>.bam.bai: sorted, indexed BAM with MM/ML, NM, RG tags; the header states that the data are simulated.
- <out>/truth_events.tsv: ground truth, one row per event (columns event, type, chrom, start, end, value).

```
event                          type        chrom  start      end        value
chr7_copy_ratio                copy_ratio  chr7   0          159345973  1.5
chr10_copy_ratio               copy_ratio  chr10  0          133797422  0.5
EGFR_high_level_amplification  copy_ratio  chr7   55014032   55207338   9.0
MGMT_promoter_methylated       methylation chr10  129466536  129467536  0.88
TERT_C228T                     snv         chr5   1295112    1295113    G>A;vaf=0.800
```

Points to check

- Mismatch rate close to 2% (above 10% with --standin: wrong --seed).
- Measured copy ratios close to the "expected" values (e.g. 0.50 for a single-copy loss).
- Hotspot VAF close to purity/2, up to sampling variance (depth ~17–35×).
- MGMT coverage above the threshold chosen for Rapid-CNS2 (10× in the launchers) and MGMT methylation ~0.88.

4.2 Classify the methylation of one sample

Aim: Obtain the methylation class (Capper et al.) that Rapid-CNS2 would assign, from a simulated BAM carrying the real profile of a sample of the target class. `[simulation/]`

Steps

1. Automatic procedure (four steps: export of the representative profile, simulation with transplant, bedMethyl, classification).

   ```
   ./run_full_mnp_classification.sh --profile astro --data-dir $DATA --ref $REF \
       --num-trees 20000 --threads 16
   ```

2. Or step by step, for instance to rerun a single step; for a BAM already simulated with --mnp-targets, only the last two commands are needed:

```
Rscript 01_export_probe_targets.R $DATA probe_targets
python3 simulate_glioma_hg38.py --profile astro --ref $REF --data-dir $DATA \
    --out sim_astro_mnp --mnp-targets probe_targets/probe_targets_astro.tsv
python3 bedmethyl_from_bam.py --bam sim_astro_mnp/SIM_ASTRO_HG38.bam \
    --out sim_astro_mnp/SIM_ASTRO_HG38.bedmethyl.bed
Rscript methylation_classification_local.R -s SIM_ASTRO_HG38 -o classif_astro \
    -i sim_astro_mnp/SIM_ASTRO_HG38.bedmethyl.bed -p $DATA/top_probes_hm450.Rdata \
    -a $DATA/HM450.hg38.manifest.gencode.v22.Rdata \
    -d $DATA/capper_top_100k_betas_binarised.Rdata -t 16 -n 20000
```

3. Number of trees: 20,000 as in production; 2,000 for quick screening (about ten times shorter, less accurate). Memory measured: 4.4 GB with 300 trees; with 2,000 trees, more than 7 GB (process killed in the test environment, which only had 7).

Files produced

- classif_<profile>/<id>_calibrated_classification.tsv: number of probes, predicted class, initial score, calibrated score.
- <id>_votes.tsv (frequencies per class), <id>_rf_details.tsv (overlaps, Brier score), <id>_methylation_hg38_HM450.RDS, <id>_probes_for_training.csv.

Points to check

- Expected class: GBM_RTK_II (gbm), A_IDH (astro), A_IDH_HG (astro grade 4), O_IDH (oligo).
- "Number of features": at most 10,000 (training probes, capped at 10,000; 10,000 measured with 6 chromosomes, 6,640 with chr17 alone); "Top_100k_overlap" (_rf_details.tsv file) close to the number of probes in the probe_targets file used.

4.3 Simulate a cohort

Aim: Produce N BAMs of distinct individuals (purity, CNV amplitude, grade, real methylation profile), then optionally measure the correct classification rate. `[simulation/]`

Steps

1. Export the real profiles: one file per individual and per class. n_per_class must equal the size of the largest group (33 for N = 99 in equal shares). If a class has fewer real samples than requested, they are reused and a warning is printed.

```
Rscript 02_export_cohort_probe_targets.R $DATA cohort_probes 33
```

2. First test a small cohort on one chromosome.

```
python3 build_cohort.py --n 9 --standin --data-dir $DATA --out-dir cohort_test \
    --chroms chr17 --jobs 3
```

3. Run the full cohort. --profile-counts imposes group sizes (sum = N); --astro-grade4-fraction turns part of the astrocytomas into grade 4 (CDKN2A/B deletion and A_IDH_HG profile).

```
python3 build_cohort.py --n 99 --ref $REF --data-dir $DATA --out-dir cohort_bams \
    --mnp-targets-dir cohort_probes --jobs 16 --cnv-jitter-sd 0.15 \
    --astro-grade4-fraction 0.3 --classify --classify-jobs 2 --num-trees 2000
```

4. Set --jobs according to memory: about 0.8 GB measured per simulation of chr17 alone, without MNP probes, after correction; use grows with the length of the largest simulated chromosome (chr1 is three times longer). A FAILED(rc=-9) status in the manifest signals a killed process, most often for lack of memory: reduce --jobs. For --classify-jobs, allow more than 7 GB per classification with 2,000 trees.

Files produced

- cohort_bams/INDnnn_<PROFILE>/: BAM, index and truth_events.tsv of each individual; INDnnn_<PROFILE>.log: log.
- cohort_bams/cohort_manifest.tsv: individual, profile, seed, purity, grade 4, expected class, probe file, BAM, status, duration.

- cohort_classification/cohort_classification_summary.tsv: expected and predicted class, calibrated score, status; correct classification rate printed at the end.

Points to check

- All rows of the manifest with status OK.
- No "no probe file found" warning: otherwise the individuals concerned have no real methylation signal.

4.4 Use the simulation interface (Streamlit)

Aim: Prepare and run procedures 4.1 and 4.3 without typing the commands. `[simulation/]`

Steps

1. Start the server on P3Ultra; from another computer, open an SSH tunnel then http://localhost:8501.

```
streamlit run simnanocns_gui_en.py
ssh -L 8501:localhost:8501 P3Ultra
```

2. Sidebar: fill in the working folder, the scripts folder (this simulation/ folder), the Python interpreter, the data/ folder and the reference. "Save as defaults" stores these values in ~/.simnanocns_gui_en.json; "Export/Import (JSON)" exchanges a configuration.
3. Single sample or Cohort tab: set the parameters. The right column shows the exact command, flags options unknown to the installed scripts and lists the checks (files present, disk space, consistency).
4. "Run" starts the command in the background; the Run tab shows the duration, the number of BAMs produced, the log and, at the end, the ground truth or the manifest and the classification rate. The simulation continues if the browser tab is closed, but stops if the Streamlit server is stopped.

Files produced

- <working folder>/gui_logs/<date>_<name>.log and .params.json: log and parameters of each run.

4.5 Run Rapid-CNS2_nf on a BAM

Aim: Analyse a simulated or real BAM with the pipeline and open its report. `[rapidcns2/]`

Steps

1. Open the interface; choose the cohort folder (the BAMs of <cohort>/*.bam and <cohort>/*/*.bam are listed) and the sample.

```
python3 rapidcns2_gui_en.py
```

2. Check the paths (main.nf, launch folder, reference, ANNOVAR, AnnotSV) and the options: Nextflow profile (local,docker or local,singularity), minimum MGMT coverage, CPUs, memory, purity for SAVANA, MNP-Flex, Human variation, resume (-resume).
3. For simulated BAMs, put in the launch folder a custom.config file that ignores the expected failures (no structural variant, no heterozygous SNP); the interface uses it automatically if it exists. Remove it for real data.

```
process {
    withName: 'annotSv.*' { errorStrategy = 'ignore' }
    withName: 'savanaCna' { errorStrategy = 'ignore' }
}
```

4. "Run" starts Nextflow; "Copy command" copies the equivalent command for a terminal. The progress bar counts the steps finished in the trace file; "Stop" stops Nextflow cleanly.
5. "Open report" opens the HTML report (full version, else lite).

Files produced

- <output>/report/<id>_Rapid-CNS2_report_full.html (and _lite.html): final report.
- <output>/mods/<id>.5mC.bedmethyl, <output>/mgmt/mgmt_avg_cov.txt, <output>/pipeline_info/*_trace.txt.

Points to check

- Compare the report with truth_events.tsv: methylation class, MGMT status, CNVs (+7/−10, EGFR, CDKN2A/B, 1p/19q), variants (IDH1, TERT, TP53).
- mgmt_avg_cov.txt above --minimumMgmtCov, otherwise the MGMT module is skipped.

4.6 Prepare real data (FASTQ to BAM)

Aim: Turn Nanopore FASTQ files into a sorted, indexed hg38 BAM usable by Rapid-CNS2. `[rapidcns2/]`

Steps

1. Check that the FASTQ files carry the MM/ML tags in their headers (produced by samtools fastq -T '*'). A standard MinKNOW FASTQ has none: methylation (classification, MGMT) will then be impossible; start again from the POD5 files or the unaligned BAM.
2. Run the alignment: one file or a folder of FASTQ files (compressed or not), the reference, the sample name; -m passes options to minimap2.

```
./align_fastq_rapidcns2_en.sh -i fastq_pass/ -r $REF -s GBM01 -o results -t 24
```

Files produced

- results/GBM01.sorted.bam (+ .bai), results/GBM01.qc.txt (flagstat, reads per chromosome, reads with MM), results/GBM01.merged.fastq.gz if a folder was merged.

Points to check

- No "no MM:Z: tag" or "contig 'chr7' missing" warning.
- In the QC file, reads carrying MM among the first 2,000 alignments.

4.7 Build and use the USB stick

Aim: Carry Rapid-CNS2 on a self-contained stick, usable offline on another Linux computer. `[usb/]`

Steps

1. Recommended method: make_usb_stick_en.sh performs all the steps below in one command, with safety checks (never a system disk) and a confirmation before erasure. The following manual steps remain valid.

```
./make_usb_stick_en.sh --list
./make_usb_stick_en.sh --dry-run /dev/sdX
./make_usb_stick_en.sh --eject /dev/sdX -- --engine apptainer --with-samples
/path/to/cohort_bams
```

2. Format the stick as ext4 (FAT32 and exFAT are refused: symbolic links, Unix permissions, files over 4 GB). This erases the stick; replug it before the chown.

```
lsblk
sudo umount /dev/sdX1
sudo mkfs.ext4 -m 0 -L RAPIDCNS2 /dev/sdX1
sudo chown $USER: /media/$USER/RAPIDCNS2
```

3. Estimate the space needed without writing anything.

```
./prepare_usb_en.sh --dry-run /media/$USER/RAPIDCNS2
```

4. Build the stick. The source paths (pipeline, reference, ANNOVAR, AnnotSV) are set by environment variables (PIPELINE_DIR, REF_FA, ANNOVAR_DIR, ANNOTSV_DIR, GUI_PY). --engine chooses apptainer, docker or both; --no-annovar excludes ANNOVAR (not redistributable); --with-samples DIR copies BAMs.

```
./prepare_usb_en.sh --engine apptainer --with-samples /path/to/cohort_bams
/media/$USER/RAPIDCNS2
```

5. On the host computer: check the installation, then start the interface or the command line. The Nextflow work folder is on the host's disk (~150 GB per sample, changeable with RCNS_WORKDIR).

```
cd /media/$USER/RAPIDCNS2/rapidcns2
./run.sh --check
./run_gui.sh
./run.sh samples/IND001_GBM/IND001_GBM.bam IND001_GBM
```

Files produced

- rapidcns2/ on the stick: bin/ (Nextflow), jre/, pipeline/, containers/, refs/, annovar/, annotsv/, gui/, samples/, results/, conf/usb.config, env.sh, run.sh, run_gui.sh, README.txt.

Points to check

- run.sh --check prints "Everything is in place." (ANNOVAR shows as missing if the stick was built with --no-annovar: then point RCNS_ANNOVARPATH and RCNS_ANNOVARDB to a local installation).
- Offline, wf-human-variation is disabled and automatic upload to MNP-Flex is impossible.
- conf/usb.config ignores AnnotSV and SAVANA failures (simulated BAMs): comment out these lines for real data.

4.8 Regenerate the data produced before the corrections

Aim: Replace the files affected by the corrected errors. `[simulation/]`

Steps

1. bedMethyl files and classifications: redo for all BAMs (– strand decoding error). The BAMs themselves remain valid.

   ```
   python3 bedmethyl_from_bam.py --bam X.bam --out X.bedmethyl.bed
   # then methylation_classification_local.R (procedure 4.2)
   ```

2. gbm BAMs intended for Rapid-CNS2: resimulate to get the new MGMT coverage (18–22×).
3. The new BAMs are not byte-identical to the former ones for the same seed (order of the methylation draws changed), but remain reproducible.

5. Detailed workings

5.1 Simulation of a BAM (simulate_glioma_hg38.py)

1. Reference: reading of the hg38 FASTA (chr* or RefSeq names) or deterministic synthetic sequence (42% GC, CpG-depleted). Check of the hotspot reference bases.
2. Tumour model: copy ratios per chromosome and focal events of the profile (table below), optionally noised (--cnv-jitter-sd: N(1, sd) factor bounded to [0.5, 1.5]).
3. Coverage plan, per chromosome: off-panel segments (off-target depth × local ratio), panel regions (on-target), dedicated targets (MGMT, ±300 bp around the hotspots and the deletion site), extra background in amplified windows, ±150 bp windows around the HM450 probes.
4. Read drawing: for each target, number of reads ~ Poisson(depth × (width + mean length) / mean length); log-normal length (mean ~3.1 kb, bounded to 300–40,000 bp; short reads of 250–2,000 bp for the probes); uniform start so that each read overlaps the target. Fragments containing N or an ambiguity code are discarded.
5. Tumour allele: each read covering a hotspot carries the mutant allele with a probability equal to the VAF; the 1 bp deletion (ATRX or CIC) is applied with probability purity/2 if it lies more than 20 bp from the read ends.
6. Strand drawn at random (50%). Sequencing errors: 4% of the bases (50% substitutions, 25% deletions, 25% insertions); CIGAR and NM rebuilt; MAPQ 60; Q20 quality.
7. Methylation: for each CpG of the read (original orientation), methylation probability from the model below, call drawn at random, ML value 200–255 if methylated, 0–55 otherwise; MM tag "C+m" with the gaps between CpG Cs.

8. Writing: reads sorted by position, indexed BAM, header with the dorado basecalling model and a "SIMULATED DATA" comment; then truth_events.tsv.

5.2 Simulated tumour profiles

Alteration	gbm	astro	oligo
Chromosome 7 / 10	gain 1.5 / loss 0.5	diploid	diploid
1p / 19q	—	—	single-copy loss (0.5)
EGFR	amplification (×9)	—	—
CDKN2A/B	homozygous deletion (0.02, ±50 kb)	grade 4 option	—
IDH1 R132H	wildtype	purity/2	purity/2
TERT C228T	0.80	wildtype	purity/2
TP53 R175H	—	purity/2	wildtype
1 bp deletion	—	ATRX	CIC
MGMT promoter	methylated (0.88)	methylated (0.88)	methylated (0.88)
Methylation background	0.47	0.64 (G-CIMP)	0.64 (G-CIMP)
Expected methylation class	GBM_RTK_II	A_IDH (A_IDH_HG in grade 4)	O_IDH

5.3 Methylation model

Site	Methylation probability
Background CpG	0.04 or 0.90 depending on a deterministic hash of the position (same value for all reads); share of sites at 0.04: 50% (gbm) or 30% (astro, oligo)
MGMT promoter	0.88
Transplanted HM450 probe	0.92 if the real sample is methylated, 0.04 otherwise
Misaligned CpG (error, insertion)	0.50

5.4 bedMethyl (bedmethyl_from_bam.py)

1. Reading of each aligned primary read carrying an MM tag; decoding of the calls by pysam (positions in the stored sequence).
2. + strand: the call is on the C; – strand: on the opposite G. The call is mapped to the position of the CpG C on the + strand; it is skipped if the C and the G are not aligned on two consecutive reference positions.
3. Aggregation per site: coverage and number of methylated calls (ML ≥ 128); writing of the 18 modkit columns, strand "." (equivalent of --combine-strands).

5.5 Classification (methylation_classification_local.R)

1. Reading of columns 1, 2, 3, 6, 10 and 11 of the bedMethyl; removal of chrX and chrY; methylation = percentage / 100.
2. Overlap with the HM450 hg38 manifest: each covered probe receives the methylation of the site.
3. Training data (Capper et al., binarised) restricted to the covered probes; the 10,000 most variable are kept.

4. Probabilistic random forest (ranger), sampling fractions balanced between classes; then one logistic regression per class to recalibrate the scores.
5. Sample binarised at 0.5, predicted class = maximum probability; writing of the initial and calibrated scores.

5.6 Verification (verify_bam.py)

1. Header and presence of MM tags.
2. Mean panel coverage per chromosome (indicative only: long reads spill over neighbouring regions).
3. Copy ratios of large events: density of off-panel reads per Mb relative to the median of simulated neutral chromosomes.
4. Focal events (EGFR, CDKN2A/B): raw coverage relative to the panel of a neutral chromosome; the direction of the signal is what matters.
5. Hotspots: A/C/G/T counts and VAF; MGMT: depth and methylation per strand; control: methylation of 40 panel regions outside MGMT.
6. Consistency with the reference: mismatch rate on 200 reads (≈ 2% expected).

5.7 Cohort (build_cohort.py)

1. Split of the N individuals between profiles (equal shares, remainder given to gbm then astro, or imposed sizes), order shuffled by the base seed.
2. For individual i: seed = base seed + i; purity drawn uniformly in --purity-range; grade 4 drawn with probability --astro-grade4-fraction (astro only); probe file of its own rank within its profile.
3. Independent simulations in parallel (--jobs), manifest; then classifications in parallel (--classify-jobs) and correct classification rate.

6. Description of the scripts and their functions

For each script: its role, then its main functions. Internal interface functions (widget building, callbacks) are grouped.

6.1 `simulation/simulate_glioma_hg38.py`

Generates a Nanopore BAM aligned to hg38 (NPHD panel, adaptive sampling) for the gbm, astro and oligo profiles, with variants, copy-number alterations and methylation (MM/ML tags), and writes the ground truth to truth_events.tsv.

Function	Role
`RealRef`	Reads a real hg38 FASTA; accepts chr1… or RefSeq (NC_000001.11…) contigs.
`StandInRef`	Deterministic synthetic sequence; can force bases (hotspots, CG of HM450 probes).
`read_bed`	Reads a BED file (chrom, start, end, name).
`build_model`	Loads panel, MGMT region and genes; builds the events of the profile.
`frameshift_site`	Site of the 1 bp deletion: middle of the largest coding exon of the longest NM_ transcript.
`ratio_at`	Copy ratio at a position (focal event takes priority).
`add_errors`	Adds the sequencing errors and returns the reference position of each base.
`cigar_from_rp`	Rebuilds the CIGAR from the reference positions.
`methylation_tags`	Computes the MM/ML tags compliant with the SAM specification.

Function	Role
`main`	Options, generation per chromosome, writing of the BAM and the ground truth.

6.2 `simulation/verify_bam.py`

Checks a simulated BAM independently of the generator and compares each measurement with truth_events.tsv.

Function	Role
`density`	Off-panel reads per Mb: copy-ratio estimator.
`mean_cov`	Mean coverage sampled over a window.
`counts`	A/C/G/T counts at a position.
`(script body)`	Successive check sections (see 5.6).

6.3 `simulation/build_cohort.py`

Simulates a cohort of N individuals in parallel, then can classify each BAM.

Function	Role
`parse_profile_counts`	Reads imposed group sizes and checks their sum.
`build_assignment`	Splits the individuals between profiles, reproducible shuffled order.
`run_one`	Simulates one individual and returns one row of the manifest.
`classify_one`	bedMethyl then R classification; reads the predicted class.
`main`	Option checks, generation, manifest, classification, summary.

6.4 `simulation/bedmethyl_from_bam.py`

Produces a modkit-format bedMethyl from the MM/ML tags (equivalent of modkit pileup --cpg --combine-strands).

Function	Role
`(linear script)`	Decoding, mapping to the + strand C, aggregation per site, writing (see 5.4).

6.5 `simulation/methylation_classification_local.R`

Local version of the Rapid-CNS2 classification (see 5.5).

Function	Role
`Reading and matching`	bedMethyl, chrX/chrY removal, overlap with the HM450 manifest.
`Training`	10,000 most variable probes; ranger balanced by class.
`Recalibration`	One-versus-all logistic regression.
`Prediction`	Binarisation, predicted class, scores and votes.

6.6 simulation/01_export_probe_targets.R / 02_export_cohort_probe_targets.R

Export real methylation profiles at the hg38 positions of the HM450 probes: one representative per class (01) or N distinct samples per class (02).

Function	Role
(linear scripts)	Intersection top_probes ∩ autosomal manifest ∩ betas; writing of the probe_targets files.

6.7 simulation/run_full_mnp_classification.sh

Complete chain for one profile (procedure 4.2).

Function	Role
(linear script)	Option validation, four steps, result display.

6.8 simulation/simnanocns_gui_en.py

Streamlit interface that builds, checks and runs the simulation commands (procedure 4.4).

Function	Role
build_single_cmds / build_cohort_cmds	Build the commands.
equal_counts / group_counts / per_class	Group sizes (largest-remainder method).
preflight / cohort_errors	Pre-launch checks.
parse_help_flags / unknown_flags	Options unknown to an older version of the scripts.
launch / stop_job / tab_run	Background launch, stop, follow-up.
Others (sidebar, tab_*, show_*)	Interface, configuration.

6.9 rapidcns2/rapidcns2_gui_en.py

Tkinter launcher for Rapid-CNS2_nf (procedure 4.5); defaults changeable with RCNS_* variables.

Function	Role
find_bams	Lists the BAMs of a cohort folder.
build_command	Builds the nextflow run command.
count_trace / task_name	Count the steps finished in the trace file.
run / _reader / _poll_log / stop	Launch, log reading, stop.
_poll_progress / _finish_progress	Progress and final state.
open_report	Opens the HTML report.

6.10 rapidcns2/align_fastq_rapidcns2_en.sh

FASTQ to sorted, indexed hg38 BAM, with checks (procedure 4.6).

Function	Role
`usage / log / warn / die`	Help and messages.
`(steps 1 to 5)`	Merge, methylation, alignment and sort, index, checks.

6.11 usb/make_usb_stick_en.sh

Creates the stick end to end: safety checks, erasure and ext4 formatting after confirmation, mounting, call of prepare_usb_en.sh, final check (procedure 4.7).

Function	Role
`run`	Runs a command, or only prints it with --dry-run.
`part1`	Name of the first partition (sdb1, nvme0n1p1, loop0p1).
`Checks`	Refuses a disk mounted outside /media, /run/media, /mnt or used as swap; requires a USB or removable disk.
`Partition and format`	wipefs, parted (or sgdisk) with GPT, mkfs.ext4 -m 0 -L RAPIDCNS2.
`Mount and end`	udisksctl or mount, chown to the user, prepare_usb_en.sh, run.sh --check, optional eject.

6.12 usb/prepare_usb_en.sh

Builds the self-contained USB stick and writes its launchers (procedure 4.7).

Function	Role
`sz / human / sif_name`	Size, display, image name in the Nextflow cache.
`rcns_check_engine / rcns_load_docker (env.sh)`	Check the container engine and load the Docker images.
`run.sh --check`	Installation check on the host.

7. Errors found and corrections

Ordered by importance. "Check" states how the correction was verified.

File	Problem	Correction	Check
`bedmethyl_from_bam.py`	– strand reads mis-decoded: the MM tag counts the Cs of the original read, the script counted those of the stored sequence. 95% of calls outside CpGs.	Decoding with pysam; calls mapped to the + strand C; non-contiguous CpGs skipped; strand ".".	– strand calls on a CpG: 5% → 76% (measured before the contiguity filter); MGMT 0.89 (expected 0.88).
`simulate_glioma_hg38.py`	Position of each base kept for every read of a chromosome: individual killed for lack of memory.	Tags computed when the read is created; ML as uint8.	chr17: 2.1 → 0.77 GB; test cohort 3/3 instead of 2/3.
`simulate_glioma_hg38.py`	Fixed MGMT depth: ~4–7× in gbm, below the 10× threshold of the launchers.	Option --mgmt-depth-factor (default 4).	MGMT 17.9× and 22.5×.

File	Problem	Correction	Check
`simnanocns_gui_en.py`	Seed not passed to verify_bam.py in stand-in mode (~74% mismatches); samtools wrongly required.	--seed passed; samtools check removed; .fai message clarified; log closed.	Generated command and checks tested.
`verify_bam.py`	Expected background hard-coded at 0.47; chr1 reference not neutral for oligo; non-simulated events shown as 0.00; samtools required.	Values read from truth_events.tsv; automatic neutral chromosome; "not simulated"; pysam.bedcov.	19q measured 0.50 (expected 0.50).
`simulate_glioma_hg38.py, build_cohort.py`	Arguments not validated: unknown chromosome → late crash; purity > 1 accepted.	Check of --chroms, --purity, --purity-range, --n, --jobs, grade 4 fraction.	Error messages tested.
`methylation_classification_local.R`	Probability ties → recalibration crash; wrong default training file; required options not checked.	Class by max.col; default fixed; checks; named scores.	Run on a simulated gbm BAM: GBM_RTK_II predicted.
`methylation_classification_local.R`	Memory: the whole bedMethyl (18 columns) and its copies stayed in memory during training; process killed at 6 GB before even reaching the random forest.	Reading of the 6 useful columns (fread); tables freed before loading the training data.	Classification completed (300 trees, 4.4 GB).
`01_ / 02_export_*.R`	Crash if a class is absent; fragile numeric conversion; arguments not checked.	Absent class skipped with a warning; as.matrix; checks.	Run: 100,000 probes per class; argument check tested.
`align_fastq_rapidcns2_en.sh`	grep -q with pipefail: possible false warning.	Contig list captured before the test.	shellcheck without warnings.
`prepare_usb_en.sh`	Interface not found after regrouping; Nextflow "-dist" binary missing from older releases; --engine none accepted.	Search in ../rapidcns2/; try -dist then -all; engines limited; argument checks.	Stick built in a sandbox + run.sh --check.
`rapidcns2_gui_en.py`	Prefixed step names misread; only the "full" report looked for.	Normalised name; fallback to "lite".	Regular expression tested.
`rapidcns2_gui_en.py, prepare_usb_en.sh (env.sh)`	With Nextflow 26.04 (strict syntax by default), Rapid-CNS2_nf 3.0 no longer starts: "Config parsing failed", import in nextflow.config, statements outside the workflow in main.nf (10 errors).	NXF_SYNTAX_PARSER=v1 set by the interface and by env.sh, unless already set; the pipeline is not modified.	Error reproduced with Nextflow 26.04.6; pipeline started with v1.
`rapidcns2_gui_en.py`	Outside the USB stick, --containerBindPaths was not passed: AnnotSV then ANNOVAR, run in Docker, did not see their folders (paths passed as text, hence not mounted). Run stopped at annotSvSeverus.	AnnotSV, ANNOVAR and humandb folders mounted automatically (no duplicates or sub-folders), completed by RCNS_BINDPATHS.	Docker options checked with Nextflow 26.04.6: -v for each folder.

File	Problem	Correction	Check
`rapidcns2_gui_en.py`	On a simulated BAM, Severus and Sniffles find no structural variant: AnnotSV exits without writing its table and Nextflow stops the whole pipeline ("Missing output file"). A custom.config had to be written by hand.	"Simulated BAM" box, ticked automatically if the BAM header contains "SIMULATED DATA"; it writes and adds .rapidcns2_simulated.config (AnnotSV and SAVANA CNA ignored, outputs unused by the report or with a fallback).	Detection tested (simulated BAM, ordinary BAM, missing file); rule checked with Nextflow 26.04.6 on annotSvSeverus, annotSvSniffles and savanaCna.
`build_cohort.py`, `run_full_mnp_classification.sh`	False warning for an empty group; blank line not handled; profile not validated.	Limited warning; robust parsing; profile checked.	Cohort of 3 individuals generated.

7.1 Points checked and left unchanged

- --savanaCnaArgs '--overrule_cellularity 0.8': Nextflow accepts a value starting with a dash if it contains a space (isValue function of the Nextflow launcher).
- All parameters passed to Rapid-CNS2_nf exist in the current version of the repository, and the tracked step names match the aliases in main.nf.
- The hg38 coordinates of the hotspots (TERT C228T, IDH1 R132H, TP53 R175H) are correct.
- The classification algorithm stays identical to that of Rapid-CNS2: only robustness fixes were made.

8. Rationale for the choices

8.1 Grouping by use

Simulation and classification stay in the same folder, because build_cohort.py, run_full_mnp_classification.sh and the Streamlit interface call these scripts by the path of their own folder: separating them would have broken these calls. The Rapid-CNS2 interface and the FASTQ alignment serve to run the pipeline; the USB stick is only for deployment.

8.2 Imitate modkit rather than invent

Rapid-CNS2_nf calls modkit pileup with --cpg --combine-strands. Reproducing this behaviour ensures that the classification receives the same kind of input as with real data. Strand "." becomes "*" in R (conversion checked in the GenomicRanges code, makeGRangesFromDataFrame) and overlaps the HM450 probes whatever their strand. The "+" used before lost half of the probes: the manifest has 49,806 of its 100,000 probes on the "−" strand.

8.3 Use pysam to decode MM/ML

Manual decoding caused the error. pysam implements the SAM specification; its behaviour was checked on the simulated BAMs (C for the + strand, opposite G for the – strand).

8.4 Memory versus exact reproducibility

Computing the tags when the read is created changes the order of the methylation draws: the BAMs remain reproducible for a given seed but are no longer byte-identical to the former ones. This trade-off was chosen because the gain (×2.7) makes many parallel simulations possible on 128 GB.

8.5 Configurable MGMT depth

The --minimumMgmtCov threshold is 5 by default in Rapid-CNS2 but 10 in the interface and the USB stick. With the former fixed value, the MGMT module risked being skipped for gbm. A factor of 4 gives 18–22×; it remains adjustable to test the pipeline at low coverage.

8.6 Validate the arguments up front

A full simulation takes a long time: a typo (unknown chromosome, out-of-range purity) must stop the script immediately with a clear message rather than midway.

8.7 Do not change the classification algorithm

The aim is to test Rapid-CNS2: the local classification must stay faithful to the pipeline's. Only failure cases were fixed.

9. Tests performed

Tests on stand-in sequence, restricted chromosomes; R scripts run on the real Rapid-CNS2 data (R 4.3, Ubuntu packages). Values measured by verify_bam.py, by reading the BAMs directly or in the classification outputs.

Check	Measured	Expected or baseline
MGMT promoter methylation (gbm)	0.89	0.88
Methylation background (gbm)	0.47	0.47
Transplanted HM450 probes recovered	296 / 300	300
Probe methylation target 0 / target 1	0.03 / 0.91	0.04 / 0.92
19q copy ratio (oligo)	0.50	0.50
CIC deletion frequency (n = 9 reads)	0.33	0.40
TP53 R175H VAF (astro, purity 0.724)	0.38	0.36
Mismatch rate	2.0 %	~2 %
MGMT coverage (gbm, two seeds)	17.9× / 22.5×	≥ 10×
Peak memory, chr17	0.77 GB	2.1 GB before correction
Test cohort (3 individuals, chr17, 3 jobs)	3/3 in 1.6 min	2/3 in 2.7 min before correction
Export 01 (representative profiles, Capper data)	100,000 probes/class in 23 s	no error
Export 02 (2 profiles per class)	8 files + manifest	no error
R classification, simulated gbm (6 chromosomes, 300 trees)	GBM_RTK_II, score 0.84	GBM_RTK_II
Classified cohort (3 individuals, chr17, real Capper profiles)	3/3 correct	GBM_RTK_II, A_IDH, O_IDH
Classification memory (300 trees)	4.4 GB	killed at 6 GB before correction
make_usb_stick_en.sh on a 6 GB virtual disk (partition, ext4, mount, content, check)	OK, then --keep-format --eject OK	OK
make_usb_stick_en.sh with the mount folder of an already formatted stick	partition found, nothing erased, stick completed	OK
make_usb_stick_en.sh on the system disk	refused, even with --allow-non-removable	refused
USB stick build + run.sh --check	OK	OK (ANNOVAR absent with --no-annovar)
Static analysis (pyflakes, shellcheck)	no error	no error

10. Limitations and recommendations

- The classification was only tested with 300 trees (2,000 exceed the memory of the test environment) and on 6 chromosomes: to be validated on P3Ultra with run_full_mnp_classification.sh, with 2,000 then 20,000 trees, for each profile.
- The recalibration emits non-convergence warnings from the logistic regression (perfect separation of the scores): they come from the Rapid-CNS2 method and do not affect the predicted class, but the calibrated score is then close to 1.
- The tests used the stand-in sequence: a run with the real hg38 (--ref) then a complete pass through Rapid-CNS2_nf are still needed.
- The whole genome was not simulated here. Partial measurement with MNP probes: 15.5 min for 6 chromosomes (chr1, 2, 7, 10, 17, 19, about a quarter of the genome) then 5.6 min for the bedMethyl; do a first run with few jobs before a large cohort.
- Depth at the hotspots remains modest (~17–35×): the measured VAF varies from one sample to the next (0.12 observed for IDH1 with 17 reads, expected 0.40). This is sampling noise; increase --ontarget-depth if a stable VAF is needed.
- The NXF_SYNTAX_PARSER=v1 workaround relies on the legacy Nextflow parser, which will be removed: Rapid-CNS2_nf will have to be migrated to the strict syntax (upstream), or Nextflow pinned to a version that keeps v1 (NXF_VER).
- make_usb_stick_en.sh was tested on a virtual (loop) disk, without udisksctl: to be tried on a real USB stick, first with --dry-run.
- The graphical interfaces (Streamlit, Tkinter) were checked for their logic, not their display.